

YouTube Channel Performance and Engagement

Rajlaxmi Kalaskar
Department of Computer
Science and Engineering
MGM's College of Engineering
Nanded, India
rajlaxmi.naresh@gmail.com

Asst. Prof. Himanshu Joshi
Department of Computer
Science and Engineering
MGM's College of Engineering
Nanded, India

Abstract— This paper presents the design and implementation of a YouTube Channel Performance and Engagement Analysis Dashboard developed using Python, Streamlit, MySQL, Pandas, Plotly, the YouTube Data API, and an optional OpenRouter-based recommendation layer. The system collects public channel and video metadata, stores the records in a relational database, calculates performance and engagement indicators, and visualizes insights through an interactive dashboard. The dashboard supports data collection from channel IDs, handles, channel links, and video links; global filtering by title, date, views, engagement level, and duration; channel profile summaries; video-level tables; trend charts; channel comparison; and AI-assisted growth strategy generation. The work demonstrates how fragmented YouTube statistics can be converted into structured, visual, and decision-oriented analytics.

Keywords— YouTube analytics, engagement rate, Streamlit dashboard, data visualization, YouTube Data API, MySQL, content performance analysis.

I. INTRODUCTION

Online video platforms generate large amounts of public performance data, but the values are often distributed across channel pages, video pages, and analytics views. For a creator, student, or analyst, manually comparing views, likes, comments, publishing dates, upload frequency, and video duration can be slow and error-prone. A dashboard provides a structured method for converting these raw values into practical insights.

The proposed YouTube Channel Performance and Engagement Analysis Dashboard is a Python-based project that collects YouTube channel and video data, stores it in MySQL, and presents it through a Streamlit interface. The application focuses on channel-level review, video-level exploration, engagement analysis, visualization, channel comparison, and strategy recommendations.

II. BACKGROUND AND RELATED WORK

YouTube analytics commonly involves studying reach, consistency, and engagement. Reach is represented through values such as views and subscribers, while engagement is reflected through likes, comments, and audience reaction. Existing platform analytics tools are useful for account owners, but students and independent analysts often need a custom system that can collect public data for multiple channels, compare channels, and demonstrate the complete data pipeline from API request to visual insight.

Python is suitable for this type of project because it provides mature libraries for API communication, data processing, database interaction, and visualization. Pandas supports tabular transformation, SQLAlchemy supports database operations, Plotly supports interactive visual charts, and Streamlit supports fast development of data applications. The YouTube Data API provides structured access to public

channel and video metadata, making it possible to build an educational analytics system without scraping web pages.

The proposed dashboard differs from a static report because it preserves collected records in a database and allows repeated exploration. A user can refresh a channel, filter records by date or duration, inspect individual video rows, and switch between trend views, ranking views, comparison views, and strategy views. This makes the system closer to a decision-support dashboard than a single-purpose visualization script.

III. PROBLEM STATEMENT

Creators and analysts need to know which videos perform well, which publishing patterns are effective, and how audience engagement changes across time. Basic public statistics such as total views and subscribers are easy to observe, but deeper interpretation requires combining multiple values. For example, engagement rate depends on likes and comments in relation to views, while content strategy depends on comparing stronger and weaker videos over time.

The project addresses the lack of a simple educational system that integrates data collection, database storage, reusable filters, visual analysis, multi-channel comparison, and recommendation support in one workflow. Instead of relying on manual spreadsheets or isolated charts, the dashboard provides a single environment for repeated analysis.

IV. OBJECTIVES

The main objective is to design and implement an interactive analytics dashboard that can collect and store public YouTube data, calculate meaningful performance metrics, visualize channel and video trends, compare channels, and produce growth-oriented recommendations. The secondary objective is to demonstrate practical use of Python libraries, API integration, relational database operations, and visualization methods in a complete application.

The project also aims to improve usability for non-technical users. The system accepts multiple YouTube input forms, gives warnings when data is unavailable, presents recent analyzed channels, and keeps navigation inside a single Streamlit sidebar. These interface choices reduce the effort required to move from raw channel input to interpretable insight.

V. METHODOLOGY

A. Data Collection

The data collection module accepts channel IDs, channel URLs, handles, custom links, usernames, video URLs, and selected search results. Inputs are resolved into canonical YouTube channel IDs. The YouTube Data API is then used to fetch channel

information, upload playlist video IDs, video metadata, statistics, thumbnails, durations, and selected top-level comments when available.

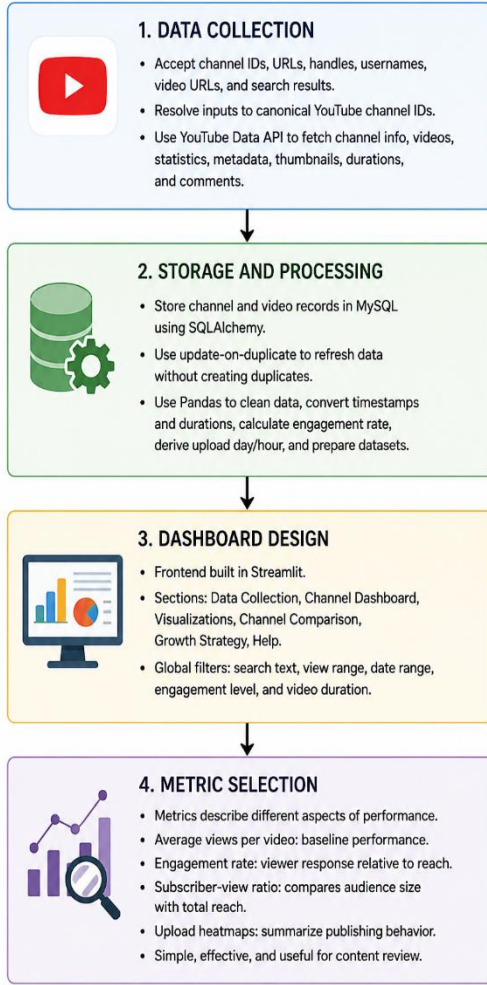


Fig. 1. Methodology of YouTube Channel Performance and Engagement

B. Storage and Processing

The project stores channel records and video records in a MySQL database using SQLAlchemy. Insert operations use update-on-duplicate behavior so that repeated data collection refreshes changing statistics without creating duplicate rows. Pandas is used to normalize API responses, convert timestamps, convert ISO 8601 durations into seconds, calculate engagement rate, derive upload day and upload hour values, and prepare chart datasets.

C. Dashboard Design

The frontend is built in Streamlit and organized into Data Collection, Channel Dashboard, Visualizations, Channel Comparison, Growth Strategy, and Help sections. Global filters allow the user to search video titles or descriptions, select view ranges, choose date ranges, filter by engagement level, and filter by video duration.

D. Metric Selection

The selected metrics were chosen because they describe different aspects of channel performance. Average views per video gives a normal performance baseline, engagement rate estimates viewer response relative to reach, subscriber-view

ratio compares audience size with total reach, and upload heatmaps summarize publishing behavior. These metrics are simple enough for academic demonstration while still being useful for content review.

Dashboard Processing Architecture

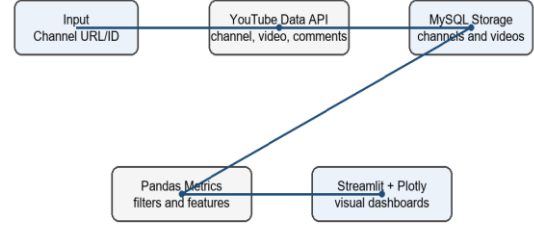


Fig. 1. System architecture of the YouTube analytics dashboard.

VI. SYSTEM IMPLEMENTATION

The dashboard module presents a channel profile card with thumbnail, channel name, description, subscribers, total views, total videos, and channel creation date. It also calculates average views per video, average engagement rate, and subscriber-view ratio. A paginated video table provides video IDs, watch links, thumbnails, titles, descriptions, publish dates, views, likes, and comments.

The visualization module provides views over time, top ten most viewed videos, yearly video distribution, views versus engagement scatter plots, upload heatmaps, view distribution histograms, engagement trend lines, engagement funnels, and custom daily, monthly, or yearly metric trends. The comparison module supports up to five channels and displays selected channel cards, leaderboard ranking, benchmark tables, bar charts, and performance share charts.

The growth strategy module identifies high-performing and low-performing videos using view-count quantiles. It extracts key phrases from video titles using TF-IDF and prepares content insights.

The data collection workflow includes several safeguards. The system resolves direct channel IDs, handles, usernames, custom links, and video links before making channel requests. Video IDs are fetched from the uploads playlist and then processed in batches, which follows the YouTube API pattern for retrieving video details. Comment collection is limited to selected videos with available comments so that the dashboard remains responsive and avoids unnecessary API calls.

The database layer separates channel-level records from video-level records. The channels table stores profile information and summary statistics, while the videos table stores metadata and engagement values for each upload. This separation avoids repeated channel details in every video row and supports future expansion, such as adding a comments table, export history, or scheduled refresh logs.

The filtering layer is shared across dashboard and visualization pages. It supports keyword search, view-count category filtering, date-range filtering, engagement-level

filtering, and duration filtering. By applying the same filter state across multiple views, the user can narrow the dataset once and then compare how the selected subset behaves in tables, charts, and metrics.

Metric	Formula/Source	Purpose
Average views	$\text{mean}(\text{view_count})$	Baseline video reach
Engagement rate	$(\text{likes} + \text{comments}) / \text{views} \times 100$	Viewer response strength
Subscriber-view ratio	$\text{subscribers} / \text{total views}$	Audience-to-reach relation
Upload heatmap	published_date grouped by day and month	Publishing pattern review

VII. RESULTS AND DISCUSSION

The implemented system successfully converts public YouTube statistics into an interactive analysis workflow. Users can collect data for a selected channel, store it persistently, apply global filters, inspect detailed video rows, study time-series charts, compare channels, and review content strategy suggestions. The dashboard is useful for identifying normal video performance, unusually strong or weak videos, upload timing patterns, and engagement behavior.

A major advantage of the system is that analysis is no longer limited to static numbers. The same stored data can be explored from different perspectives such as trend analysis, top-video ranking, engagement funnel review, duration filtering, publish-date filtering, and channel benchmarking..

The channel dashboard provides a concise summary for one selected channel, while the visualization page supports deeper exploration. For example, a views-over-time chart can reveal whether a channel has seasonal growth or a few isolated spikes. A top-ten chart highlights the videos that contribute most to reach. A views-versus-engagement chart can reveal whether popular videos also receive strong viewer response or whether some smaller videos attract more active audiences.

The comparison page is useful when several channels belong to the same niche or academic case study. Leaderboards and benchmark tables make it easier to identify which channel has stronger subscriber count, total views, or upload volume. The performance share chart gives a quick visual estimate of how total views are distributed across selected channels. Limiting comparison to five channels keeps the interface readable and prevents charts from becoming crowded.

The growth strategy page adds an interpretive layer to the dashboard. High-performing and low-performing videos are separated using quantiles, and title phrases are extracted to identify topic patterns. This does not replace human judgment, but it helps users begin analysis with evidence from their own dataset rather than generic advice. The local fallback strategy also ensures that the feature remains usable even when the external AI model cannot be called.

The current limitation is that results depend on the availability and quota limits of the YouTube Data API. Some videos may have unavailable likes, disabled comments, or missing metadata. The project also focuses on descriptive analytics rather than predictive modeling. Future work can include sentiment analysis of comments, scheduled data refresh, exportable PDF or Word reports, predictive view estimation, and deeper audience behavior analysis.

VIII. CONCLUSION

The YouTube Channel Performance and Engagement Analysis Dashboard demonstrates a complete analytics workflow using Python, Streamlit, MySQL, Pandas, Plotly, and the YouTube Data API. It provides data collection, storage, filtering, metric calculation, visualization, channel comparison, and recommendation features in one application. The project shows how public video-platform data can be transformed into structured insights that help users understand performance, engagement, and content strategy.

ACKNOWLEDGMENT

The authors would like to express their gratitude to the research community for the open-access datasets and pre-trained Transformer models that made the development of the YouTube Channel Performance and Engagement platform possible. Special thanks are extended to the peer reviewers for their insightful comments, which significantly improved the technical rigor of this manuscript. Furthermore, the authors acknowledge the institutional support provided for the computational resources utilized during the experimental evaluation phase.

REFERENCES

- [1] Google Developers, “YouTube Data API v3,” Google, documentation. [Online]. Available: <https://developers.google.com/youtube/v3>. Accessed: May 2026.
- [2] Streamlit, “Streamlit Documentation,” Streamlit Inc., documentation. [Online]. Available: <https://docs.streamlit.io/>. Accessed: May 2026.
- [3] Plotly Technologies Inc., “Plotly Python Graphing Library,” documentation. [Online]. Available: <https://plotly.com/python/>. Accessed: May 2026.
- [4] W. McKinney, Python for Data Analysis, 3rd ed. Sebastopol, CA, USA: O’Reilly Media, 2022.
- [5] Pandas Development Team, “Pandas Documentation,” documentation. [Online]. Available: <https://pandas.pydata.org/docs/>. Accessed: May 2026.
- [6] SQLAlchemy Authors, “SQLAlchemy Documentation,” documentation. [Online]. Available: <https://docs.sqlalchemy.org/>. Accessed: May 2026.
- [7] MySQL, “MySQL Reference Manual,” Oracle Corporation, documentation. [Online]. Available: <https://dev.mysql.com/doc/>. Accessed: May 2026.
- [8] Scikit-learn Developers, “scikit-learn User Guide,” documentation. [Online]. Available: https://scikit-learn.org/stable/user_guide.html. Accessed: May 2026.